Feature Selection & Dimensionality Reduction

Code Companion — Running the Examples

This companion walks you through the runnable Python scripts that accompany the guide *Feature Selection & Dimensionality Reduction — A Beginner's Field Guide*. Each script is self-contained, runs on the two included datasets, and prints its results so you can compare against what you read. This booklet shows you how to set everything up, then explains what each script does and what output to expect.

> **What you'll need**
> Python 3.10 or newer, and an internet connection the first time (to download the libraries). Everything else — the scripts and both datasets — is already in the package. No prior command-line experience assumed.

1. What's in the package

After you unzip `feature_selection_code.zip`, you'll have this layout:

```
feature_selection_code/
  README.md              <- quick reference
  requirements.txt       <- the libraries to install
  run_all.py             <- runs all ten scripts at once
  datasets/
     house_price.csv        (regression examples)
     bike_buyers_clean.csv  (classification examples)
  scripts/
     common.py           <- shared data loaders
     01_multicollinearity_vif.py
     02_variance_threshold.py
     ... through ...
     10_full_pipeline_workflow.py
```

The two datasets let every script run offline. **house_price.csv** (21,613 homes, all-numeric) drives the regression examples; **bike_buyers_clean.csv** (1,000 customers, mixed types) drives the classification examples.

2. Setup (one time)

These steps use Windows Command Prompt, assuming Python is installed at `C:\python3_13` and you extract the package to `C:\ml_projects`. Adjust paths to match your machine. (macOS / Linux users: the same steps work — use forward slashes and `source .venv/bin/activate` in step 3.)

Step 1 — Extract the package

```
mkdir C:\ml_projects
tar -xf %USERPROFILE%\Downloads\feature_selection_code.zip -C C:\ml_projects
```

Or right-click the zip in File Explorer, choose *Extract All*, and point it at `C:\ml_projects`.

Step 2 — Create a virtual environment

A **virtual environment** is an isolated package area for this project, so its libraries don't touch your system Python. Create one called `.venv` inside the project:

```
cd C:\ml_projects\feature_selection_code
C:\python3_13\python.exe -m venv .venv
```

Step 3 — Activate it

```
.venv\Scripts\activate.bat
```

Your prompt now shows `(.venv)` at the front — that means the environment is active. Type `deactivate` when you're done to leave it.

Step 4 — Install the libraries

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

> **Where do the files go?**
> `pip` downloads scikit-learn, pandas, numpy, and statsmodels from PyPI (about 60-90 MB, one to two minutes). The installed packages land inside your project at `.venv\Lib\site-packages` — nothing installs into `C:\python3_13`. Downloaded files are cached under `%LOCALAPPDATA%\pip\Cache` for reuse.

Coming back later

You create the venv only once. In a fresh Command Prompt, just reactivate before running scripts:

```
cd C:\ml_projects\feature_selection_code
.venv\Scripts\activate.bat
```

3. Running the scripts

With the environment active, move into the `scripts` folder and run any script by name:

```
cd scripts
python 01_multicollinearity_vif.py
```

Run them in numeric order for the full tour, or jump to the one matching the guide section you're reading. To run all ten at once and get a pass/fail summary, use the helper from the project root:

```
python run_all.py
```

> **Reproducibility**
> Random splits use a fixed seed, so your numbers should match on re-runs. Mutual-information and cross-validation results can vary slightly across scikit-learn versions — the selected features and overall story stay the same.

4. The scripts at a glance

Script	Guide §	What it shows
01_multicollinearity_vif	2	VIF; spotting redundant / perfectly-overlapping features

Script	Guide §	What it shows
02_variance_threshold	3.1	Dropping near-constant features (filter)
03_f_statistic_regression	3.2	f_regression + SelectKBest (linear links)
04_mutual_information	3.3	Mutual information; the non-linear case the F-test misses
05_chi2_classification	3.4	chi2 on categories; f_classif for contrast
06_embedded_ridge_lasso	4	Ridge + SelectFromModel; Lasso auto-selection
07_sequential_selection	5.1	Forward & backward SequentialFeatureSelector
08_rfe_rfecv	5.2	RFE and the safer RFECV (in a pipeline)
09_pca_dimensionality	6	PCA explained-variance curve; component counts
10_full_pipeline_workflow	7	End-to-end leak-free pipeline + cross-validation

5. What each script does

A short walkthrough of every script: the idea it demonstrates, and a sample of the output you'll see. Your numbers should closely match.

01 — Multicollinearity (VIF)

Computes the Variance Inflation Factor for each numeric feature. High VIF means a feature is largely explained by the others. This dataset contains a perfect relationship — `sqft_living = sqft_above + sqft_basement` — so those three show an *infinite* VIF, the exact perfect-multicollinearity idea from section 2.

```
Features with VIF > 10 (strong overlap): ['sqft_living',
  'sqft_basement', 'sqft_above', 'lat', 'long', 'grade', ...]
Perfect multicollinearity (infinite VIF): ['sqft_living',
  'sqft_basement', 'sqft_above']
  -> e.g. sqft_living = sqft_above + sqft_basement; drop one.
```

02 — Variance threshold

Splits into train/test, fits `VarianceThreshold` on the training set only, and drops columns whose variance is below the cutoff. Notice it's applied before any scaling.

```
Original feature count : 17
Kept after threshold   : 13
Dropped features: ['waterfront', 'lat', 'long', 'renovated']
```

03 — F-statistic (f_regression)

Ranks features by their linear relationship to price, then keeps the top 8 with `SelectKBest`. The strongest match the F-regression figure in the guide.

```
      Feature      F_score       p_value
  sqft_living 16981.142859  0.000000e+00
        grade 13542.457721  0.000000e+00
   sqft_above 10119.199735  0.000000e+00
  ...
Top 8 features selected:
['bedrooms','bathrooms','sqft_living','waterfront',...]
```

04 — Mutual information

Part A builds a synthetic U-shaped relationship: the F-test scores it near zero (misses it) while MI scores it clearly positive (catches it). Part B ranks the real features by mutual information.

```
U-shaped feature vs target:
  F-test score :    0.952   (near 0 -> F-test misses it)
  MI score     :    2.798   (positive -> MI catches it)
```

> **This is the key lesson of MI**
>
> Same data, two verdicts: the linear F-test sees no relationship in the U-shape, but mutual information detects the dependence. When relationships may be curved, MI is the safer scorer.

05 — Chi-squared (classification)

Switches to the bike-buyers data. Ordinal-encodes the categorical columns (chi2 needs non-negative values) and scores each against whether the customer bought a bike. Also runs `f_classif` on the numeric columns for contrast.

```
         Feature  Chi2_score  p_value
       Education   17.788499   0.0000
  Marital Status    7.961891   0.0048
Commute Distance    3.048854   0.0808
  ...
Top 50% categorical features by chi2:
['Marital Status', 'Education', 'Commute Distance']
```

06 — Embedded (Ridge & Lasso)

Both models live in a `StandardScaler` pipeline so scaling is fit on training folds only. Ridge shrinks coefficients and we select the strongest with `SelectFromModel`; Lasso zeroes weak coefficients on its own.

```
RidgeCV chose alpha = 100
Ridge-selected features (|coef| >= median): ['bedrooms',
  'bathrooms', 'sqft_living', 'waterfront', 'view', ...]
LassoCV chose alpha = 262.2
Features kept (non-zero coef): 16 of 17
```

07 — Sequential feature selection

Runs forward selection (add best feature each round) and backward selection (remove worst), each with an explicit scoring metric and 5-fold cross-validation. Here both directions agree.

```
Forward-selected : ['bedrooms','bathrooms','sqft_living',
  'waterfront','view','grade','lat','age']
Backward-selected: ['bedrooms','bathrooms','sqft_living',
  'waterfront','view','grade','lat','age']
```

08 — RFE and RFECV

Plain RFE keeps a fixed number of features; RFECV lets cross-validation choose how many. RFECV is the safer default because it validates the count for you.

```
Plain RFE  -> Selected: ['Income', 'Children', 'Cars']
RFECV      -> Optimal number of features: 4
           -> Selected: ['Income','Children','Cars','Age']
```

09 — PCA dimensionality reduction

Standardizes, fits PCA, and prints the cumulative explained-variance curve so you can pick a component count. It reports how many components reach 90% and 95% — and reminds you that explained variance is not the same as predictive information.

```
  10 components:  89.5%
  11 components:  92.5%
  ...
Components to reach 90% variance: 11
Reduced data shape: (21613, 17) -> (21613, 11)
```

10 — Full pipeline workflow

The capstone: a single pipeline chaining scaling, selection, and a model, evaluated with cross-validation. Because selection happens *inside* the pipeline, the scores are honest out-of-sample estimates. It compares all features against the top-8 subset.

```
5-fold cross-validated RMSE (lower is better):
  All 17 features : 203,889
  Top 8 features  : 228,189
```

Why the smaller set scored worse here

Feature selection doesn't always improve prediction — on this dataset the full set wins. That's a real and useful result: selection buys simplicity, speed, and interpretability, and the honest CV comparison tells you when the trade-off is worth it. The point is to *measure*, not assume.

6. Troubleshooting

Symptom	Likely cause & fix
'python' is not recognized	The venv isn't active, or Python isn't on PATH. Re-run the activate step, or call it by full path (C:\python3_13\python.exe).
ModuleNotFoundError: sklearn	Libraries not installed in the active venv. Confirm you see (.venv) in the prompt, then run pip install -r requirements.txt again.
SSL / connection error on pip	You're behind a proxy or firewall. Set HTTP_PROXY / HTTPS_PROXY, or ask your network admin; then retry the install.
FileNotFoundError for a .csv	Run scripts from inside the scripts folder. The loaders resolve datasets relative to the package, so keep the folder structure intact.
Numbers differ slightly	Different scikit-learn version. Small differences in MI or CV scores are normal; the selected features should match.
ConvergenceWarning (if it appears)	Harmless for these demos — the model hit its iteration cap but still gives usable coefficients. The scripts already quiet the common ones.

Verifying your install

The fastest end-to-end check is the run-all helper. A clean run ends with a line like this:

```
Done: 10/10 scripts passed.
```

If you see that, your environment is set up correctly and every example is working.

Feature Selection & Dimensionality Reduction — Code Companion

Part of the AI Foundations Series